

Implementing the Hangman Game

In this coursework, your task is to implement a version of the classic word-guessing game commonly known as “Hangman”. The game needs to be playable using a graphical user interface (GUI) implemented in Java. The deadline for this coursework is Friday April 5th (5pm local time).

1. Hangman – Description of the Game

Hangman is a word-guessing game, meaning that the player’s goal is to guess a word. At the beginning of the game a word is selected for the player to guess. The game is played in a series of rounds.

Each round proceeds as follows:

1. The player is presented with the current information they have about the word — the letters they have guessed so far are shown, and in place of the letters not yet guessed a dash is displayed. (In particular, this means that in the first round of the game, all that the player sees is a series of dashes, one for each letter of the word they are guessing.)
2. The player guesses a letter.
3. If the guessed letter appears in the word, the player is informed of the correct guess by replacing the corresponding dashes in the word’s display with the guessed letter.
4. If the guessed letter does not appear in the word, the player is informed that the guess was incorrect.¹

The player has a limited number of incorrect guesses they are allowed to make before guessing the word. The player wins by guessing all the letters of the word before using up the allowed incorrect guesses or lives if you will. If the player reaches the limit of the incorrect guesses before guessing the word, they lose the game. You can see a screen shot of what this could look like at the end of the document and you can find more detailed information on the game itself here: [https://en.wikipedia.org/wiki/Hangman_\(game\)](https://en.wikipedia.org/wiki/Hangman_(game)).

2. Coursework Specification

The goal is to implement a single-player version of Hangman with a graphical interface.² The program picks the word, and the player is trying to guess it.

2.1. Mandatory Features

This is the list of the mandatory features. For the submission to be considered complete, all of the following have to be implemented. You can find specific weekly tasks in section 3.

- The program has to implement the hangman game for guessing words in the English language.
- At startup, the program will read a list of words from a file. Make sure there are **no limitations on the number of words** your program can support. Any limitations on the supported file format need to be clearly documented (e.g., one word per line, only uppercase letters). For testing your program, you can use the example word list files provided in appendix B.

¹Traditionally, this is done by adding an element to a drawing of a hanged man. This is where the game’s name comes from.

²The emphasis of this coursework is on the interface but in order to get the full marks, you will need to implement the functionality as well. Please, refer to the rubric in the end of the document.

- At startup, the program will display the game GUI and start a new game.
- Whenever a new game is started, the word to be guessed will be picked by selecting a random word from the list that was loaded at startup.
- The game's GUI has to have the following features:
 - Display of the word being guessed showing the guessed letters and a placeholder symbol (e.g., a dash) in place of the letters that are to be guessed.
 - Buttons for guessing the letters — one button per letter. The interface has to prevent the user from making the same guess multiple times (e.g., by disabling the button or making it invisible after it was used once).
 - Visual indicator showing the player how many incorrect guesses they have made so far and how many they have remaining before they lose the game. The player has to be allowed to make at least three incorrect guesses. Beyond that, feel free to decide how difficult you want to make the game. The fewer mistakes you allow the player, the more difficult the game.
- If the player wins the game, the program will congratulate the player and ask them if they want to play another game. If the player decides to play another game, the program will start a new game. If the player decides not to play another game, the program will terminate its execution.
- If the player loses the game, the program will inform the player of the defeat, revealing the word the player was guessing, and asking them if they want to play another game. If the player decides to play another game, the program will start a new game. If the player decides not to play another game, the program will terminate its execution.
- If at any time the player closes the main GUI window, the program will terminate its execution.

There is an example GUI design and a screen capture of the game run in appendix A. The example is there to showcase the mandatory features. **You do not have to implement the exact same interface.** You are free to decide on the exact presentation and design of the GUI elements, as long as you follow the basic principles of GUI design and OOP.

2.2. Optional Features

In addition to the mandatory features listed above, you might want to try to implement some of the following features. These are only suggestions and will not award you any additional marks.

- Provide an interface element for changing the number of incorrect guesses allowed before the game is lost. Perhaps even remember this setting, so that when the program is terminated and started again, the last set setting is loaded.
- Provide an interface element for changing the word list being used.
- Allow the player to specify what kind of word would they like to be guessing (e.g., give me a 6-letter word).
- Make the game playable in multiple languages.
- Implement the option to play the game in single player or multiplayer mode, where in multiplayer mode two players with their own respective health bars can be playing turn-by-turn.

If you think of any additional features you would like to implement, feel free to do so. However, make sure that your submission *still covers all the mandatory features*.

3. Development Schedule and Code Organization

Like for the first coursework, you will have weekly or bi-weekly tasks. The deadlines mentioned here are only suggestions but you will need to have at least one commit on GitLab for each of the tasks specified below. *If you do not have this commit history, you will receive 0 marks for this coursework.* Start by forking the following repository.

https://gitlab-student.macs.hw.ac.uk/f27sb_2023-24/f27sb_2023-24_coursework/gui_coursework

Note: The provided Run class is the only place of the program that should have a main method. This class should not contain anything else but the main method and the main method should not contain any code apart from the creation of an instance of your main window and the definition of the title, size, and visibility of this main window.

3.1. Week 7-8 — suggested deadline: end of week 8

1. Implement a new class `MainWindow` which inherits from `JFrame` and will be the window where you will show all the components of the game.
2. Create a class called `HealthPanel` which should inherit from `JPanel`. This GUI element displays the number of incorrect guesses the player has made and the remaining amount of incorrect guesses that can be made before the game is lost.
 - a) This class should contain a number of labels equal to the amount of incorrect guesses allowed. To begin with, the labels should be green.
 - b) This class needs a method called `removeLife` which will be called to update the element once an incorrect guess has been made. One of the green labels should be turned red. The return value of the method can indicate if the player has run out of their allotment of incorrect guesses.
3. Add your `HealthPanel` to the `MainWindow`.

3.2. Week 9 — suggested deadline: end of week 9

1. Develop the capability to load the list of words from a file and the capability to select a random word from that list.
2. Implement a class `WordPanel` which inherits from `JPanel`. This class should contain GUI elements that allow you to display the guess prompt for the user, i.e., displaying the word to be guessed, with the letters that have not been guessed replaced by some symbol.
 - a) The constructor should accept a `String` of the word that is to be guessed. Based on this word, the text on a label in this class should be changed to show a number of underscores equal to the number of letters of the word.
 - b) This class needs a method called `guess` which accepts a `String` of the letter that the user has guessed and is called when a guess is made. The method changes the text of the label if the guess is correct by replacing the corresponding underscore(s) with the guessed letter. The return value of the method should indicate if the guess is successful or not.
 - c) There should also be a method for checking if the word is completely guessed.
3. Add the `WordPanel` to the `MainWindow`.

3.3. Week 10 - 11 — suggested deadline: end of week 11

1. Create a class called `ButtonPanel` which inherits from `JPanel`. This should contain all the buttons of all the letters of the English Alphabet. Hence, there will have to be 26 buttons in total. Do *not* code each one individually! Have an array of buttons. Associate event handlers with the buttons so that when a button is clicked, the corresponding letter can be extracted. Once a button has been pressed it should be disabled or made invisible.
 - a) Add the `ButtonPanel` to the `MainWindow`.
 - b) Find a way to call the `guess` method of the `WordPanel` from within the event handler of the buttons so that the guessed letter can be put into the method.
2. Finalise the game mechanic. Whenever a button is pressed, it needs to be checked if the player guessed the entire word correctly or has run out of lives. Choose the best place to do all this in your program based on convenience but also cohesion and coupling.
3. At the end of the game
 - a) Implement the display of the player's victory or defeat messages. Provide a way for the player to either play again or to end the game. You can do all this via dynamic interface changes or via Dialogue boxes. Make sure that upon defeat, the entire word is revealed to the player.
 - b) Implement the functionality to reset the interface and start a new game with a newly selected random word.

The deadlines are suggestions. Feel free to use this list to help you organise your development, and keep in mind that you will need to make at least one commit per task mentioned. If you would like to deviate from the class structure that is given here, you are welcome to do so. There are numerous ways one can approach this task, and you are encouraged to explore your own ideas. However, keep in mind cohesion and coupling.

4. Final submission

The final deadline is **April 5th (Friday) at 5pm (local time)**. For your final submission, please submit a link to your fork of the program on Canvas. Additionally, please download a zip file of your code from GitLab and upload it to Canvas as well. To determine if a submission was on time, only the submission on Canvas counts.

5. Marking Scheme

This coursework is worth **[15 marks]**. The division of marks is given below.

- **[2 marks]** A panel (e.g. `HealthPanel`) displaying the number of incorrect guesses and the number of attempts remaining.
- **[3 marks]** A panel (e.g. `ButtonPanel`) consisting of the English alphabet as buttons which deactivate or disappear upon being pressed.
- **[3 marks]** A panel (e.g. `WordPanel`) displaying the word to be guessed, incrementally revealing the correctly guessed letters.

- **[4 marks]** The overall game consisting of the three interacting panels, along with an appropriate message appearing (allowing to end the game or start a new one) upon completion of a round. Full marks will be given for correctly implemented game logic as described in section 2.1.
- **[3 marks]** Code quality. For full marks you need to have a tidy, well-structured, and well-commented codebase which follows all the guidelines about good design, coupling, and cohesion from the lectures.

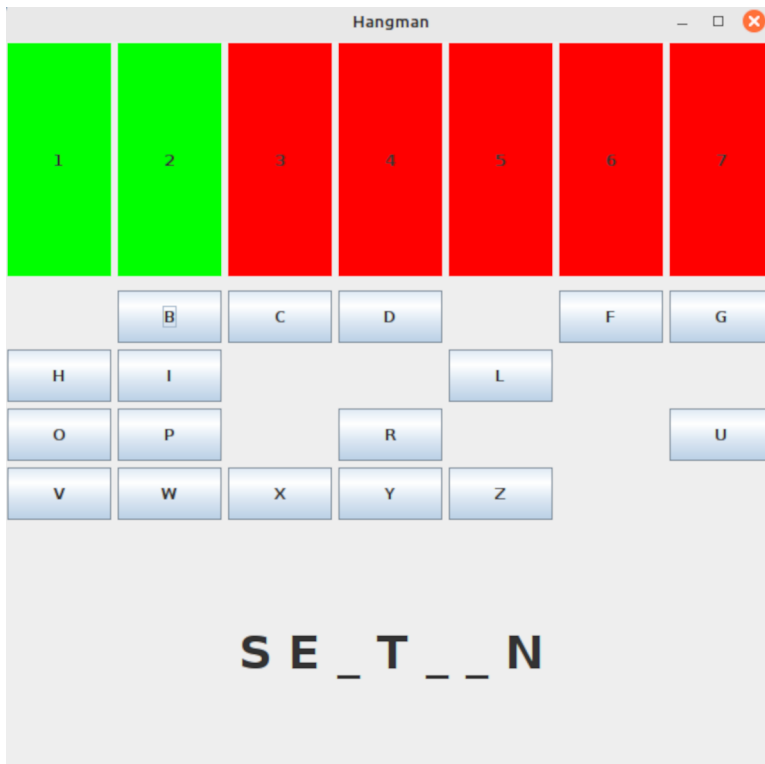
As mentioned earlier, you are welcome to deviate from the suggested class structure, in which case you will be marked appropriately based on your work. You can then consider the above allocation of marks an approximate weighting of the necessary components of your program.

6. Class Test

There will be a class test in Week 12 based on the work you have had to do in this coursework. In this class test you will be asked questions about the three panels comprising the Hangman game you have developed. In particular, this means you should consider all the tasks in section 3.1 and 3.2 along with task 1 in section 3.3 to be *examinable*.

A. Example Application Design

Here is an example of a possible GUI design. Please note that you do not have to copy this exact design.



A panel with seven labels. Each green label represents an available incorrect guess. As an incorrect guess is made, the rightmost green label turns red.

A panel with 26 buttons, one for each letter of the English alphabet. Buttons corresponding to the already made guesses become invisible.

A label displaying the player's goal. Correctly guessed letters are displayed on their positions, and the letters that have not yet been guessed correctly are replaced by underscores.

Here is a short video showing the game with the above interface in action: https://www.macs.hw.ac.uk/~md2021/teaching/F27SB/CW2/gameplay_example.mp4. (You may need to download the mp4 file first if your browser has trouble playing it.)

B. Example Word Lists

- A list of 1098 nouns, obtained from <https://eslgrammar.org/list-of-nouns/>:
 - <https://www.macs.hw.ac.uk/~md2021/teaching/F27SB/CW2/nouns.txt>
- A list of 349296 words, obtained from <https://github.com/dwyl/english-words>:
 - <https://www.macs.hw.ac.uk/~md2021/teaching/F27SB/CW2/words.txt>